

Cold Review of the `ALCI_RCC5` Lean Development Rounds 19–25

Adversarial adequacy review of definitions and theorem statements

14 July 2026

Abstract

This report reviews the uploaded artifact `Round19Transport.lean` against the neutral reference semantics in `ALCI_RCC5_REFERENCE.md`. It does not re-check kernel proofs. The target is adequacy: whether the formal statements mean `ALCI_RCC5` concept satisfiability and whether the certificate/catalogue interface is non-vacuous and finite enough to support the advertised reduction.

The bottom line is **gap, repairable**. I agree that the defects appear fixable and local to the modelling/reduction interface rather than signs that the kernel-checked proofs are invalid. I do *not* regard all of them as minor relative to the advertised decidability story: the soundness pipeline is credible, but the finite-certificate/completeness interface needs substantive tightening before it can honestly support a decision-procedure claim. This is not a second mathematical conjecture beyond bounded-width/F3 completeness; it is a formalization-level objection that the current Lean statement does not yet express that assumption as a bounded, finite, computably checkable obligation.

1 Scope and standard of review

The submitted source says it is the normative artifact and that the prose companions are commentary. I used `ALCI_RCC5_REFERENCE.md` as the yardstick. Line numbers in this report refer to the file as archived in `cold_review_round25.zip`.

The review question is not whether Lean accepted the proofs. It is whether the Lean definitions and theorem statements match:

- the standard RCC5 atomic-network semantics with strong equality;
- NNF `ALCI` syntax with RCC5 atoms as roles and inverse roles absorbed by converse;
- concept satisfiability as nonempty extension of the input concept;
- a finite and computably searchable certificate/catalogue reduction.

2 Verdict

Verdict: gap, repairable.

The core soundness chain is largely adequate:

`Wellformed+SCond+Hintikka` $\implies$ `certInterp_rcc5` $\implies$ `truth_lemma` $\implies$ `Satisfiable`.

I found no polarity error in `sat`, no missing NNF constructor in `Concept`, and no apparent RCC5 composition-table error.

The gap is in the reduction interface. The current formalization proves a sound machine from generated objects to models. It does not yet formalize an honest finite, bounded, decidable search space for those generated objects. The most important defects are:

1. `Cert`, `Template`, and `Catalog` contain arbitrary higher-order function fields, so they are not finite syntactic certificates as Lean objects.
2. `Satisfiable` is monomorphic over the countable occurrence carrier `Occ`, whereas the reference semantics quantifies over arbitrary nonempty domains of regions.
3. `CompletenessObligation` is an unbounded existence proposition, not a finite bounded search theorem.
4. `SCat.net_r3` is over-strong on degenerate template triples and can reject semantically equivalent certificates because of unused diagonal template values.

These are not proof failures; they are definitional and statement-level gaps. They are repairable, but they are substantive for the advertised reduction to decidability.

Relation to bounded-width/F3. The finite-certificate finding should not be read as a new open mathematical assumption in addition to the bounded-width/F3 completeness claim. The latter remains the main unproved mathematical step: every satisfiable concept should have a bounded generated certificate of the advertised kind. The formalization problem is that the current `CompletenessObligation` does not yet state that step in a decision-grade form. It quantifies over higher-order catalogue objects and an unbounded labelling rather than over finite codes bounded by a computable function of the input concept. Thus the defect is serious for the claim that the Lean artifact has isolated exactly one decision-ready assumption, but it is not evidence of an additional RCC5 obstruction.

3 Summary table

The IDs in the following table are referee finding labels only; in particular, Finding F3 is unrelated to the project’s bounded-width/F3 terminology.

ID	Location	Finding	Severity
F1	<code>Template</code> ; <code>Cert</code> ; <code>Catalog</code> ; <code>CatOk.F_reads</code> ; <code>Completeness-</code> <code>Obligation</code>	Certificates/catalogues are represented by arbitrary total and higher-order functions, not finite syntax.	Fatal to finite-decision claim; repairable
F2	<code>Interp</code> ; <code>Satisfiable</code>	Satisfiability is over subdomains of <code>Occ</code> , not arbitrary reference domains.	Major adequacy gap; repairable
F3	<code>Completeness-</code> <code>Obligation</code>	Obligation lacks computable bounds, finite code, closure restriction, and decidability theorem.	Fatal to current reduction claim; repairable
F4	<code>SCat.net_r3</code> versus <code>SCond.tri</code>	Catalogue check quantifies degenerate triples involving the fresh port itself; semantic condition excludes them.	Moderate over-restriction; repairable
F5	<code>certC</code> ; <code>certK</code> ; <code>sample-</code> <code>satisfiable_ex</code>	Positive witnesses are mostly all-DR and do not exercise hard positive composition/catalogue cases.	Evidence gap, not fatal

F6	Headline theorem statements	Main soundness statements are mostly honest; some prose overstates hypotheses or finiteness.	Mostly adequate; wording repairs
----	-----------------------------	--	----------------------------------

4 Findings

4.1 F1. Catalog and Cert are not finite certificates

Locations.

- `Template.net`: lines 224–228.
- `Cert.coreNet` and `Cert.f`: lines 234–241.
- `Catalog.coreNet` and `Catalog.F`: lines 3311–3317.
- `CatOk.F_reads`: lines 3505–3514.
- `CompletenessObligation`: lines 4459–4476.

The artifact repeatedly uses finite-certificate language, but the Lean structures contain fields of the following shape:

```

structure Template where
  nSlots : Nat
  nFresh : Nat
  net : TPort -> TPort -> Atom

structure Cert where
  nCore : Nat
  coreNet : Nat -> Nat -> Atom
  templates : List Template
  steps : List Step
  f : Nat -> (Nat -> Atom) -> (Nat -> Atom) -> Nat -> Atom

structure Catalog where
  nCore : Nat
  coreNet : Nat -> Nat -> Atom
  templates : List Template
  root : Nat
  attaches : List Attach
  F : Nat -> (Nat -> Atom) -> (Nat -> Atom) -> Nat -> Atom

```

These fields are arbitrary functions. The field `F` is higher-order in row functions. The later predicates constrain finite slices of their behavior, but the witnesses themselves are not finite codes. In particular, `CatOk.F_reads` is an extensionality discipline over arbitrary row functions, not a finite lookup table.

Why this matters. A Lean proof of `CompletenessObligation` could choose a function `F` that hides an infinite or noncomputable construction. Even if all queries made by `SCat` and `Hintikka` are finite for a fixed object, the object being existentially quantified is not itself a finite certificate. Thus the current statement does not by itself support finite search.

Concrete symptom. Two catalogues may agree on every value ever inspected by a given plan and `Hintikka` labelling, while differing arbitrarily on unused template indices, unused ports, unused

fresh indices, or row-function arguments. The formalization has no finite normal form or quotient identifying these as the same certificate.

Required repair. Introduce finite encoded syntax and make total evaluators derived from it. A repair can take either of these forms:

1. Replace functional fields by bounded finite tables, for example

```
structure FinTemplate where
  nSlots : Nat
  nFresh : Nat
  netTable : Array Atom      -- indexed by bounded ports

structure FinCatalog where
  nCore : Nat
  coreNetTable : Array Atom  -- indexed by Fin nCore x Fin nCore
  templates : Array FinTemplate
  root : Fin templates.size
  attaches : Array FinAttach
  FTable : Array Atom        -- indexed by template, row-code, row-code, fresh
                              index
```

and define `evalTemplate`, `evalCatalog`, `evalF` as total functions from these finite codes.

2. Or keep the existing functional structures only as semantic views, but add a separate finite-code predicate:

```
structure EncodedByFiniteCode (K : Catalog) (code : FinCatalogCode) : Prop := ...
```

and require `CompletenessObligation` to produce such a code, with a computable size bound.

In both approaches, `F_reads` should become a theorem about a finite row-code evaluator, not an unrestricted higher-order assumption.

4.2 F2. Satisfiable fixes the carrier to Occ

Locations.

- Reference semantics: `ALCI_RCC5_REFERENCE.md`, lines 50–55 and 70–73.
- `Interp`: lines 4129–4135.
- `Satisfiable`: lines 4280–4282.

The reference interpretation is $I = (\Delta, \rho, \cdot^I)$, where Δ is an arbitrary nonempty domain of regions. The Lean structure is:

```
structure Interp where
  dom : Occ -> Prop
  rho : Occ -> Occ -> Atom
  val : Nat -> Occ -> Prop

def Satisfiable (C0 : Concept) : Prop :=
  exists I : Interp, RCC5Interp I /\ exists x, I.dom x /\ sat I x C0
```

Thus satisfiability is quantified only over subdomains of the countable carrier `Occ`. This is plausibly harmless for ordinary ALC-style concept satisfiability if one proves a generated-submodel/countable-model property. An ALCI model can usually be restricted to the root and

recursively chosen existential witnesses; universal restrictions are downward-preserved, and RCC5 R1/R2/R3 are inherited by induced substructures. But that theorem is not stated here.

Required repair. Use a carrier-polymorphic interpretation:

```
structure Interp (alpha : Type) where
  dom : alpha -> Prop
  rho : alpha -> alpha -> Atom
  val : Nat -> alpha -> Prop

def ReferenceSatisfiable (C0 : Concept) : Prop :=
  exists alpha, exists I : Interp alpha,
    RCC5Interp I /\ exists x, I.dom x /\ sat I x C0
```

Then prove that the certificate semantics embeds into this reference predicate. Alternatively, keep `Occ` for the generated model but add an explicit bridge theorem:

```
theorem reference_satisfiable_iff_occ_satisfiable (C0 : Concept) :
  ReferenceSatisfiable C0 <-> Satisfiable C0 := ...
```

Until that bridge exists, the Lean target predicate is an under-approximation of the reference semantics.

4.3 F3. CompletenessObligation is not a finite decision statement

Location. `CompletenessObligation`, lines 4459–4476.

The obligation says that every `Satisfiable` concept has some catalogue, plan, list labelling, and root satisfying the generated soundness interface. It does not assert any of the following:

- a computable bound on the number of templates, core nodes, slots, fresh ports, plan length, or row codes;
- that catalogues and certificates are finite encoded data rather than arbitrary functions;
- that `tau` ranges only over the closure of C_0 ;
- that `CatOk`, `PlanOk`, `SCat`, and `Hintikka` are decidable over the encoded search space;
- a theorem of the form `Decidable (ReferenceSatisfiable C0)` or an executable Boolean decision procedure with correctness proof.

The source comment at lines 4451–4458 mentions the finite type space $\leq 2^{|closure(C_0)|}$, but this bound is not part of the proposition. As stated, `CompletenessObligation` is a semantic existence theorem over higher-order Lean objects. It can close a semantic equivalence, but not by itself a finite search reduction. In the project’s terminology, this is best understood as an underformalized bounded-width/F3 interface, not as an additional independent completeness conjecture.

Required repair. Replace or supplement `CompletenessObligation` with a bounded finite version. A suitable shape is:

```
def CompletenessBounded : Prop :=
  forall C0, ReferenceSatisfiable C0 ->
    exists code : FinCatalogCode C0,
      code.size <= bound C0 /\
      Decodable code /\
      let K := decodeCatalog code
      exists planCode tauCode rootCode,
```

```

planCode.size <= planBound C0 /\
tauCode.closedUnder (closure C0) /\
CatOk K /\ PlanOk K (decodePlan planCode) /\
SCat (buildCert K (decodePlan planCode)) /\
Hintikka ... /\ rootCarries C0

```

Then add the actual decision theorem:

```

theorem decidable_reference_satisfiable (C0 : Concept) :
  Decidable (ReferenceSatisfiable C0) := ...

```

or, preferably, an executable checker:

```

def decideSat (C0 : Concept) : Bool := ...

theorem decideSat_correct (C0 : Concept) :
  decideSat C0 = true <-> ReferenceSatisfiable C0 := ...

```

This is the most important repair for the advertised decidability claim.

4.4 F4. SCat.net_r3 over-checks degenerate template triples

Locations.

- SCond.tri: lines 1861–1866.
- SCat.net_r3: lines 2797–2802.

The semantic S-condition excludes all degenerate triples through a fresh occurrence:

```

SCond.tri : ...
x != y -> x != born i jz -> y != born i jz ->
pairVal C x y in comp (pairVal C x (born i jz))
                      (pairVal C (born i jz) y)

```

The catalogue check has only $p \neq q$:

```

SCat.net_r3 : ...
p in memberPortsList C i -> q in memberPortsList C i ->
j < nFresh -> p != q ->
net p q in comp (net p fresh_j) (conv (net q fresh_j))

```

It does not require $p \neq \text{fresh}_j$ or $q \neq \text{fresh}_j$. Hence it can inspect $\text{net fresh}_j \text{ fresh}_j$, a diagonal template value that is not used as an off-diagonal semantic relation. This makes SCat strictly stronger than the intended local R3 check unless template diagonals are normalized.

Concrete calculation. Consider one template with two fresh ports and no core/slots. Set:

```

net fresh0 fresh1 = PP
net fresh1 fresh0 = PPI
net fresh0 fresh0 = DR
net fresh1 fresh1 = DR

```

Converse coherence holds for these entries. Semantically, SCond.tri has no nondegenerate triple among only two fresh occurrences. But SCat.net_r3, with $p = \text{fresh}_0$, $q = \text{fresh}_1$, and $j = 1$, demands:

```

PP in comp PP (conv DR)

```

Since `conv DR = DR` and `comp PP DR = [DR]`, this is false. Thus `SCat` rejects an object for an irrelevant diagonal choice.

Required repair. Use one of the following fixes.

1. Align `SCat.net_r3` with `SCond.tri` by adding side conditions:

```
p != inr (inr j) -> q != inr (inr j) ->
net p q in comp (net p fresh_j) (conv (net q fresh_j))
```

2. Alternatively, require normalized template diagonals:

```
net_diag : forall t p, validPort t p -> (K.tmpl t).net p p = EQ
```

and include this in `CatOk/Wellformed`. Then make explicit that templates are total atomic networks including diagonal EQ, not just off-diagonal ledgers.

The first repair is semantically minimal. The second is also acceptable if normalization is intended.

4.5 F5. Current positive witnesses are not rich enough

Locations.

- `certC`: lines 1657–1664.
- `certK`: lines 3995–4003.
- `sample_satisfiable_ex`: lines 4483–4534.

The provided positive certificate/catalogue witnesses use `nCore := 0` and all-DR template/-core/steering data. They exercise plumbing and inherited slot paths but not the hard positive cases: PP/PPI transitivity, PO cases, core rows, nontrivial steering, and non-DR successful composition. `sample_satisfiable_ex` does exercise a real existential, but it is a hand-built two-point interpretation, not a generated certificate plus Hintikka labelling passed through `sat_from_hintikka`.

This is not evidence of vacuity. A simple rich positive certificate should be available: three fresh occurrences with

$$0 \text{ PP } 1, \quad 1 \text{ PP } 2, \quad 0 \text{ PP } 2,$$

converses in the opposite direction and diagonal EQ. This exercises `comp PP PP = [PP]` positively.

Required repair. Add at least one generated positive witness with:

- three fresh points and a PP-chain using `comp PP PP`;
- at least one PO case;
- at least one inherited slot;
- nonzero core or an explicit explanation why core-free certificates suffice for the witness suite;
- a root-anchored Hintikka labelling discharged through `sat_from_hintikka` or `generated_satisfiable`.

5 Adequacy checks with no defect found

5.1 Syntax

Location. `Concept`, lines 4117–4127.

The constructors match the reference grammar in NNF:

```
top | bot | atom | natom | and | or | ex Atom | all Atom
```

There is no general negation constructor, which correctly builds the NNF discipline into the syntax. Roles are exactly RCC5 atoms; inverse roles are absorbed by converse, so $PP^- = PPI$ is representable without a separate inverse-role constructor.

5.2 Satisfaction

Location. `sat`, lines 4137–4146.

The role reading and quantifier polarity are correct:

```
| x, ex r c => exists y, I.dom y /\ I.rho x y = r /\ sat I y c
| x, all r c => forall y, I.dom y -> I.rho x y = r -> sat I y c
```

This matches the reference reading $R(x, y)$ iff $\rho(x, y) = R$. The guard is on the quantified element y . The only caveat is that `sat I x C` is defined for any $x : \text{Occ}$; theorem statements that use it as model truth carry a domain hypothesis, so this is not a soundness bug.

5.3 RCC5 interpretation frame

Location. `RCC5Interp`, lines 4206–4214.

The fields encode reflexive EQ, strong EQ as identity, converse coherence, and R3 composition closure:

```
refl_eq : dom x -> rho x x = EQ
eq_id   : dom x -> dom y -> rho x y = EQ -> x = y
conv_   : dom x -> dom y -> rho y x = conv (rho x y)
comp_   : dom x -> dom y -> dom z ->
          rho x z in comp (rho x y) (rho y z)
```

The R3 clause is stated for all triples, not just distinct triples, but the EQ identity rows make the degenerate cases harmless in actual interpretations. Standalone `RCC5Interp` permits an empty domain; `Satisfiable` supplies the root witness.

5.4 RCC5 table

Location. `conv` and `comp`, lines 138–159.

The converse table and the relevant composition entries agree with the standard RCC5 atomic composition table, including:

```
comp PP PP   = [PP]
comp PP DR   = [DR]
comp DR PPI  = [DR]
comp PPI PP  = [EQ, PP, PPI, PO]
comp PO PO   = all atoms
comp DR DR   = all atoms
```

I found no table-orientation defect.

6 Headline theorem statement audit

6.1 frame_closed

Location. Lines 2291–2300.

The theorem is conditional on operational frame entries existing:

```
Frame.get? (unfoldAll C) x y = some v1
Frame.get? (unfoldAll C) x z = some v2
Frame.get? (unfoldAll C) z y = some v3
```

Therefore it is not by itself a theorem that the unfolded frame is a total atomic network. This is acceptable because later model construction uses `pairVal`; the prose should not overstate `frame_closed` as a total-frame closure theorem.

6.2 scat_scond

Location. Lines 3205–3227.

The statement is good:

```
Wellformed C -> SCat C -> Faithful C -> SCond C
```

It does not smuggle in `SCond`. The caveat is the over-strong `SCat.net_r3` clause described in F4.

6.3 catalogue_soundness

Location. Lines 3946–3958.

The theorem is sound but has extra hypotheses beyond a slogan like “`SCat` alone yields `SCond`”:

```
CatOk K -> hcc -> hcr -> PlanOk K plan ->
SCat (buildCert K plan) -> SCond (buildCert K plan)
```

The converse-coherence hypothesis `hcc` and core-row hypothesis `hcr` should either be folded into `CatOk` or named as part of the generated-catalogue wellformedness predicate.

6.4 build_wellformed and build_faithful

Locations. `build_wellformed`, line 3708; `build_faithful`, line 3815.

`build_wellformed` has the expected form:

```
CatOk K -> PlanOk K plan -> Wellformed (buildCert K plan)
```

`build_faithful` also requires global `hcc/hcr` hypotheses. This is not unsound, but those hypotheses are part of the finite catalogue data and should be included in the eventual encoded check.

6.5 truth_lemma and sat_from_hintikka

Locations. `truth_lemma`, lines 4173–4197; `sat_from_hintikka`, lines 4292–4300.

These statements are clean. `sat_from_hintikka` is the central soundness theorem:

```
Wellformed C -> SCond C -> Hintikka dom rho tau ->
root in domain -> C0 in tau root -> Satisfiable C0
```

It neither assumes the conclusion nor hides the root witness.

6.6 `model_hintikkaP` and `satisfiable_iff_hintikkaP`

Locations. Lines 4383–4405.

These are correct but weak relative to the prose phrase “type-system certificate loses nothing”. The forward direction of `satisfiable_iff_hintikkaP` chooses:

```
tau x C := sat I x C
```

So the theorem proves unrestricted semantic Hintikka-realizability, not finite closure-bounded type realizability, not generated-certificate realizability, and not a decision procedure. The theorem is useful as a semantic abstraction lemma, but it should not be advertised as finite type-system completeness.

6.7 `generated_satisfiable`

Location. Lines 4428–4449.

This is a sound composition theorem: a generated catalogue/plan satisfying the listed conditions and carrying a root-anchored `Hintikka` labelling produces `Satisfiable C0`. It is not itself a completeness theorem; it is the target interface for the missing generator.

7 Required repairs as a consolidated checklist

1. **Separate reference semantics from generated semantics.** Define carrier-polymorphic `ReferenceSatisfiable`, or prove an explicit countable/`Occ`-model theorem bridging arbitrary reference models to the generated carrier.
2. **Finite-code the certificate and catalogue syntax.** Replace arbitrary `Nat -> Nat -> Atom`, `TPort -> TPort -> Atom`, and higher-order `F` fields by finite arrays or finite codes, with total evaluators and proofs that all semantic views are decoded from those codes.
3. **Strengthen CompletenessObligation.** State a bounded obligation that produces finite codes of size bounded by computable functions of C_0 , closure-bounded type labels, and finite plan/root codes. This is the place where the bounded-width/`F3` mathematical assumption should be formalized. Add a proved checker or `Decidable`-theorem for the bounded search space.
4. **Fix `SCat.net_r3`.** Add `p != fresh_j` and `q != fresh_j`, or require normalized diagonal EQ for all valid template ports and document that templates are total atomic networks with diagonal EQ.
5. **Fold global catalogue coherence into `Cat0k`.** The hypotheses `hcc` and `hcr` occur in `build_faithful`, `catalogue_soundness`, `generated_satisfiable`, and `CompletenessObligation`. They should be part of a single decidable finite catalogue-wellformedness predicate.
6. **Add rich non-vacuity witnesses.** Include generated positive certificates exercising non-DR composition, PP/PPI chains, PO, inherited slots, and a real Hintikka existential discharged through the certificate pipeline.
7. **Revise prose claims and theorem comments.** Keep the strong claim for the soundness pipeline. Qualify `satisfiable_iff_hintikkaP` as semantic Hintikka completeness, not finite type-system completeness. State explicitly that the current `CompletenessObligation` is not yet the finite bounded obligation needed for decidability.

8 What survives the review

The important positive result is that the formalized soundness theorem looks semantically meaningful. A wellformed, S-conditioned certificate with a Hintikka labelling yields an RCC5 interpretation and a root satisfying the input concept. The syntax and satisfaction clauses line up with the reference logic. The RCC5 frame predicate is the right strong-EQ atomic-frame notion.

Thus the defects do not suggest that the kernel has checked a vacuous proof of an obviously wrong satisfaction relation. They instead identify the missing finite and reference-semantics bridges needed before the artifact can honestly claim a finite decidability reduction.

A Accompanying scripts

The package contains the following scripts.

- `scripts/audit_static.py`: extracts source locations and flags the higher-order fields and theorem signatures discussed above.
- `scripts/scat_degenerate_check.py`: computes the concrete RCC5 table calculation behind F4.
- `scripts/build_report.sh`: recompiles this PDF from the TeX source.
- `scripts/optional_lean_check.sh`: optionally invokes Lean on `Round19Transport.lean` when Lean is installed.